

DEEP LEARNING

Numericals, Notes & Solved Examples

IIT Madras | Prof. Lakshmi Narasimhan T

All Chapters: 1 through 8 — Complete Coverage

- Ch 1: ML Fundamentals — Vectors, Bayes, Loss, Regression
- Ch 2: MLP — Activation Functions, UAT, Expressivity
- Ch 3: Training — Backprop, SGD, Adam, Dropout, BN
- Ch 4: CNNs — Convolution, Pooling, AlexNet to ResNet
- Ch 5: RNNs — BPTT, LSTM, GRU, Bidirectional
- Ch 6: Attention — QKV, Multi-Head, Transformers
- Ch 7: Autoencoders — DAE, CAE, VAE with Numericals
- Ch 8: State-of-Art — GNNs, PINNs, KANs, Diffusion

Every Topic Has: Theory + Worked Numericals + Exam Tips

Machine Learning Fundamentals

This chapter covers the mathematical bedrock of deep learning: linear algebra, probability theory, supervised learning, loss functions, and model selection. Every concept is illustrated with full numerical examples.

1.1 Vectors, Inner Products & Norms

A **vector** is an ordered list of numbers. In deep learning, every data sample (e.g., an image flattened to pixels) is a vector.

KEY CONCEPT

Inner Product (Dot Product)

$$x^T y = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

Measures how much two vectors "point in the same direction."

L2 Norm (Euclidean length): $\|x\|_2 = \sqrt{x^T x} = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$

L1 Norm: $\|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$

Lp Norm (general): $\|x\|_p = (|x_1|^p + \dots + |x_n|^p)^{1/p}$

SOLVED EXAMPLE

Problem: Given $x=[3,4]$ and $y=[1,-2]$, find: (a) $x^T y$, (b) $\|x\|_2$, (c) $\|y\|_1$, (d) angle between x and y .

(a) Inner product:

$$x^T y = 3 \cdot 1 + 4 \cdot (-2) = 3 - 8 = -5$$

Since $x^T y < 0$, vectors point in somewhat opposite directions.

(b) L2 norm of x :

$$\|x\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9+16} = \sqrt{25} = 5$$

(c) L1 norm of y :

$$\|y\|_1 = |1| + |-2| = 1 + 2 = 3$$

(d) Angle θ between x and y :

$$\cos(\theta) = x^T y / (\|x\|_2 \cdot \|y\|_2)$$

$$\|y\|_2 = \sqrt{1+4} = \sqrt{5} \approx 2.236$$

$$\cos(\theta) = -5 / (5 \cdot 2.236) = -5 / 11.18 \approx -0.4472$$

$$\theta = \arccos(-0.4472) \approx 116.57 \text{ degrees}$$

The vectors are obtuse ($>90^\circ$), confirming opposite direction.

SOLVED EXAMPLE

Problem: Are vectors $a=[2, 3, 0]$ and $b=[-3, 2, 5]$ orthogonal?

$$a^T b = 2*(-3) + 3*2 + 0*5 = -6 + 6 + 0 = 0$$

Yes! Since $a^T b = 0$, they are orthogonal (perpendicular).

1.2 Hyperplanes & Distance

A hyperplane in R^n is an $(n-1)$ dimensional flat subspace. In 2D it is a line; in 3D a plane. They are the decision boundaries of linear classifiers.

KEY CONCEPT

Hyperplane equation: $w^T x - w_0 = 0$ (w is the normal vector, w_0 is the offset)

$$\text{Distance of point } x \text{ from hyperplane} = (w^T x - w_0) / ||w||_2$$

SOLVED EXAMPLE

Problem: Hyperplane: $3x_1 + 4x_2 - 5 = 0$. Find distance of point $p=[1, 2]$ from it.

$$w = [3, 4], w_0 = 5$$

$$w^T p = 3*1 + 4*2 = 3 + 8 = 11$$

$$||w||_2 = \sqrt{9+16} = 5$$

$$\text{Distance} = (w^T p - w_0) / ||w||_2 = (11 - 5) / 5 = 6/5 = 1.2$$

Point is 1.2 units away from the hyperplane, on the positive side ($w^T p > w_0$).

1.3 Bayes' Theorem

KEY CONCEPT

$$P(X|Y) = P(Y|X) * P(X) / P(Y)$$

Prior $P(X)$: belief before seeing data. **Likelihood $P(Y|X)$:** probability of data given X . **Posterior $P(X|Y)$:** updated belief after seeing data.

SOLVED EXAMPLE

Problem (Medical Test): Disease prevalence = 1% (prior). Test sensitivity = 99% ($P(\text{Test+}|\text{Disease})$). Specificity = 95% ($P(\text{Test-}|\text{No Disease})$). If test is positive, what is $P(\text{Disease}|\text{Test+})$?

$$P(D) = 0.01, P(\text{no } D) = 0.99$$

$$P(T+|D) = 0.99, P(T+|no D) = 1 - 0.95 = 0.05$$

$$P(T+) = P(T+|D)*P(D) + P(T+|noD)*P(noD)$$

$$= 0.99*0.01 + 0.05*0.99 = 0.0099 + 0.0495 = 0.0594$$

$$P(D|T+) = P(T+|D)*P(D) / P(T+)$$

$$= 0.0099 / 0.0594 \approx 0.1667 = 16.67\%$$

Despite 99% sensitivity, only ~17% chance of disease! Low prevalence dominates.

1.4 KL Divergence & Cross-Entropy

KEY CONCEPT

$$KL(P||Q) = \sum_x P(x) * \log(P(x)/Q(x))$$

$$\text{Cross-Entropy } H(P;Q) = -\sum_x P(x)*\log(Q(x)) = KL(P||Q) + H(P)$$

Minimizing cross-entropy (used in neural network training) is equivalent to minimizing KL divergence between predictions and true distribution.

SOLVED EXAMPLE

Problem: $P=[0.4,0.6]$, $Q=[0.5,0.5]$. Compute $KL(P||Q)$ and Cross-Entropy $H(P;Q)$.

$$KL(P||Q) = 0.4*\log(0.4/0.5) + 0.6*\log(0.6/0.5)$$

$$= 0.4*\log(0.8) + 0.6*\log(1.2)$$

$$= 0.4*(-0.2231) + 0.6*(0.1823)$$

$$= -0.0893 + 0.1094 = 0.0201$$

$$H(P) = -(0.4*\log(0.4) + 0.6*\log(0.6))$$

$$= -(0.4*(-0.916) + 0.6*(-0.510)) = -(-0.366-0.306) = 0.673$$

$$H(P;Q) = KL(P||Q) + H(P) = 0.0201 + 0.673 = 0.693$$

Note: $H(P;Q) = \log(2) \approx 0.693$ for uniform Q , as expected.

1.5 Loss Functions — Detailed Numericals

KEY CONCEPT

MSE (Regression): $L = (1/N) * \sum (y_i - \hat{y}_i)^2$

MAE: $L = (1/N) * \sum |y_i - \hat{y}_i|$

Binary Cross-Entropy: $L = -[y*\log(p) + (1-y)*\log(1-p)]$

Multi-class CE: $L = -\sum_k y_k * \log(p_k)$

SOLVED EXAMPLE

Problem: True labels $y=[1,0,1,1]$, predictions $p=[0.9,0.2,0.6,0.3]$. Compute Binary Cross-Entropy loss.

$$\text{Sample 1: } y=1, p=0.9 \rightarrow L1 = -[1 \cdot \log(0.9) + 0 \cdot \log(0.1)] = -\log(0.9) = 0.1054$$

$$\text{Sample 2: } y=0, p=0.2 \rightarrow L2 = -[0 \cdot \log(0.2) + 1 \cdot \log(0.8)] = -\log(0.8) = 0.2231$$

$$\text{Sample 3: } y=1, p=0.6 \rightarrow L3 = -\log(0.6) = 0.5108$$

$$\text{Sample 4: } y=1, p=0.3 \rightarrow L4 = -\log(0.3) = 1.2040$$

$$\text{Average BCE} = (0.1054 + 0.2231 + 0.5108 + 1.2040) / 4 = 2.0433 / 4 = 0.5108$$

Sample 4 ($y=1$ but $p=0.3$) contributes most — highly wrong prediction.

SOLVED EXAMPLE

Problem (Softmax + Multi-class CE): 3-class problem. Logits $z=[2.0, 1.0, 0.5]$. True class = 1 (one-hot $y=[1,0,0]$). Compute softmax probabilities and CE loss.

$$\exp(z) = [\exp(2.0), \exp(1.0), \exp(0.5)] = [7.389, 2.718, 1.649]$$

$$\text{sum} = 7.389 + 2.718 + 1.649 = 11.756$$

$$\text{softmax} = [7.389/11.756, 2.718/11.756, 1.649/11.756]$$

$$= [0.6285, 0.2312, 0.1403]$$

$$\text{CE Loss} = -[1 \cdot \log(0.6285) + 0 \cdot \log(0.2312) + 0 \cdot \log(0.1403)]$$

$$= -\log(0.6285) = 0.4644$$

1.6 Linear Regression — Analytical Solution

KEY CONCEPT

Model: $y = w^T x + b$ (or $y = w^T x$ with augmented x)

MSE Loss: $L(w) = ||y - Xw||^2 / N$

Optimal weights: $w^* = (X^T X)^{-1} X^T y$ [Normal equations]

This is the closed-form solution — valid only for linear regression.

SOLVED EXAMPLE

Problem: Fit $y = w_1 x + w_0$ to data: $x=[1,2,3,4]$, $y=[2,4,5,4]$.

Design matrix X (with bias column):

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 4 \end{bmatrix}^T$$

$$X^T X = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 2+4+5+4 & 1*2+2*4+3*5+4*4 \end{bmatrix} = \begin{bmatrix} 15 & 39 \end{bmatrix}$$

$$(X^T X)^{-1}: \det = 4*30 - 10*10 = 120 - 100 = 20$$

$$(X^T X)^{-1} = (1/20) * \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}$$

$$w^* = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix} * \begin{bmatrix} 15 \\ 39 \end{bmatrix}$$

$$= \begin{bmatrix} 1.5*15 + (-0.5)*39 & (-0.5)*15 + 0.2*39 \end{bmatrix}$$

$$= \begin{bmatrix} 22.5 - 19.5 & -7.5 + 7.8 \end{bmatrix} = \begin{bmatrix} 3.0 & 0.3 \end{bmatrix}$$

So $w_0 = 3.0$ (intercept), $w_1 = 0.3$ (slope). Model: $y = 0.3x + 3.0$

Predictions: $[3.3, 3.6, 3.9, 4.2]$ vs True: $[2, 4, 5, 4]$.

1.7 Bias-Variance Tradeoff

KEY CONCEPT

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible_Noise}$$

Bias: avg error from wrong assumptions (underfitting → high bias).

Variance: sensitivity to small fluctuations in training data (overfitting → high variance).

SOLVED EXAMPLE

Problem: Model trained 3 times on different datasets gives predictions on test point x^* : $[2.1, 1.9, 2.3]$. True value = 3.0. Compute bias and variance.

Mean prediction = $(2.1+1.9+2.3)/3 = 6.3/3 = 2.1$

Bias = $E[y_{\text{hat}}] - y_{\text{true}} = 2.1 - 3.0 = -0.9 \rightarrow \text{Bias}^2 = 0.81$

Variance = $E[(y_{\text{hat}} - \text{mean})^2]$

= $[(2.1-2.1)^2 + (1.9-2.1)^2 + (2.3-2.1)^2] / 3$

= $[0 + 0.04 + 0.04] / 3 = 0.0267$

MSE $\approx \text{Bias}^2 + \text{Variance} = 0.81 + 0.027 = 0.837$

High bias (underfitting): model consistently predicts far from truth.

Multilayer Perceptron (MLP)

The MLP is the core building block of deep learning. We cover activation functions with derivatives, full forward-pass calculations, and expressivity analysis.

2.1 Activation Functions — Formulas & Derivatives

Function	Formula	Derivative	Range	Issue
Sigmoid	$\sigma(x)=1/(1+e^{-x})$	$\sigma(x)(1-\sigma(x))$	$(0,1)$	Vanishing grad for $ x \gg 0$
Tanh	$\tanh(x)=(e^x-e^{-x})/(e^x+e^{-x})$	$1-\tanh(x)^2$	$(-1,1)$	Vanishing grad
ReLU	$\max(0,x)$	1 if $x>0$ else 0	$[0,\infty)$	Dying ReLU ($x<0$ always 0)
Leaky ReLU	$\max(\alpha x, x) \alpha=0.01$	1 if $x>0$ else 0.01	$(-\infty,\infty)$	α is a hyperparameter
Softmax	$e^{x_i}/\sum e^{x_j}$	$p_i(1-p_i)$ (diagonal)	$(0,1), \Sigma=1$	Expensive for large K
SELU	$\lambda(x \text{ if } x>0 \text{ else } \alpha(e^x-1))$	$\lambda \text{ if } x>0 \text{ else } \lambda \alpha e^x$	$(-\lambda\alpha,\infty)$	Needs LeCun init

SOLVED EXAMPLE

Problem: Compute sigmoid, tanh, and ReLU outputs for $x = \{-2, 0, 1, 3\}$.

$x = -2$:

$$\text{sigmoid}(-2) = 1/(1+e^2) = 1/(1+7.389) = 0.1192$$

$$\tanh(-2) = (e^{-2}-e^2)/(e^{-2}+e^2) = (0.135-7.389)/(0.135+7.389) = -0.9640$$

$$\text{ReLU}(-2) = \max(0, -2) = 0$$

$x = 0$:

$$\text{sigmoid}(0) = 1/2 = 0.5$$

$$\tanh(0) = 0$$

$$\text{ReLU}(0) = 0$$

$x = 1$:

$$\text{sigmoid}(1) = 1/(1+e^{-1}) = 1/1.368 = 0.7311$$

$$\tanh(1) = (e-e^{-1})/(e+e^{-1}) = (2.718-0.368)/(2.718+0.368) = 0.7616$$

$$\text{ReLU}(1) = 1$$

$x = 3$:

$$\text{sigmoid}(3) = 1/(1+e^{-3}) = 1/1.0498 = 0.9526$$

$$\tanh(3) = 0.9951$$

$$\text{ReLU}(3) = 3$$

SOLVED EXAMPLE

Problem: Compute derivative of sigmoid at $x=1$ and verify numerically.

$$\sigma(1) = 0.7311$$

$$d\sigma/dx|_{x=1} = \sigma(1) * (1 - \sigma(1)) = 0.7311 * 0.2689 = 0.1966$$

$$\text{Numerical check: } [\sigma(1.001) - \sigma(1.000)] / 0.001$$

$$\sigma(1.001) = 1 / (1 + e^{-1.001}) \approx 0.7313$$

$$[0.7313 - 0.7311] / 0.001 = 0.0002 / 0.001 \approx 0.197 \checkmark$$

SOLVED EXAMPLE

Problem (Softmax): Compute softmax for logits $z=[1.0, 2.0, 0.5]$.

$$\exp(z) = [e^{1.0}, e^{2.0}, e^{0.5}] = [2.718, 7.389, 1.649]$$

$$\text{sum} = 2.718 + 7.389 + 1.649 = 11.756$$

$$\text{softmax}(z) = [2.718/11.756, 7.389/11.756, 1.649/11.756]$$

$$= [0.2312, 0.6285, 0.1403]$$

Sum = 1.0 $\checkmark$ Class 2 ($z=2.0$) has highest probability.

2.2 Complete Forward Pass — Full Numerical Example

KEY CONCEPT

Architecture: $2 \rightarrow 3 \rightarrow 1$ network. Sigmoid hidden, sigmoid output.

For each layer: $z = Wx + b$, then $a = \text{activation}(z)$

SOLVED EXAMPLE

Problem: Input $x=[0.5, 0.3]$. Weights and biases below. Compute output.

$$\text{Layer 1 weights } W_1 (3 \times 2): [[0.2, 0.8], [-0.4, 0.6], [0.7, -0.3]]$$

$$\text{Layer 1 bias } b_1 (3): [0.1, -0.2, 0.05]$$

$$\text{Layer 2 weights } W_2 (1 \times 3): [[0.5, -0.6, 0.4]]$$

$$\text{Layer 2 bias } b_2 (1): [0.1]$$

Layer 1 — compute z_1 :

$$z_1[0] = 0.2 * 0.5 + 0.8 * 0.3 + 0.1 = 0.10 + 0.24 + 0.10 = 0.44$$

$$z_1[1] = -0.4 * 0.5 + 0.6 * 0.3 + (-0.2) = -0.20 + 0.18 - 0.20 = -0.22$$

$$z_1[2] = 0.7 * 0.5 + (-0.3) * 0.3 + 0.05 = 0.35 - 0.09 + 0.05 = 0.31$$

Layer 1 — apply sigmoid $a1 = \sigma(z1)$:

$$a1[0] = \sigma(0.44) = 1/(1+e^{\{-0.44\}}) = 1/1.6434 = 0.6082$$

$$a1[1] = \sigma(-0.22) = 1/(1+e^{\{0.22\}}) = 1/2.2459 = 0.4452$$

$$a1[2] = \sigma(0.31) = 1/(1+e^{\{-0.31\}}) = 1/1.7033 = 0.5769$$

Layer 2 — compute $z2$ and output:

$$z2 = 0.5*0.6082 + (-0.6)*0.4452 + 0.4*0.5769 + 0.1$$

$$= 0.3041 - 0.2671 + 0.2308 + 0.1 = 0.3678$$

$$\text{output} = \sigma(0.3678) = 1/(1+e^{\{-0.3678\}}) = 1/1.6953 = 0.5899$$

Final prediction: 0.5899 (if threshold 0.5, classify as class 1)

EXAM TIP**Exam Tips for Forward Pass:**

- Always compute z first (linear), then apply activation.
- For softmax output, compute all exp values before dividing.
- Double-check matrix dimensions: W is $(n_{\text{out}} \times n_{\text{in}})$.

Training Neural Networks

This is the most exam-heavy chapter. We cover backpropagation from scratch, all optimizers with step-by-step calculations, regularization numerically, and normalizations.

3.1 Gradient Descent — Step-by-Step

KEY CONCEPT

$$w_{\{n+1\}} = w_n - \text{eta} * dL/dw$$

eta (learning rate): step size. Gradient dL/dw : slope of loss w.r.t. weight.

Intuition: Climb down the loss surface by taking small steps opposite to the slope.

SOLVED EXAMPLE

Problem: $L(w) = (w-3)^2$. Start at $w_0=0$, $\text{eta}=0.1$. Run 5 gradient descent steps.

$$dL/dw = 2(w-3)$$

Step 0: $w=0.0$, $dL/dw=2(0-3)=-6$, $w_1 = 0.0 - 0.1*(-6) = 0.6$

Step 1: $w=0.6$, $dL/dw=2(0.6-3)=-4.8$, $w_2 = 0.6 - 0.1*(-4.8) = 1.08$

Step 2: $w=1.08$, $dL/dw=2(1.08-3)=-3.84$, $w_3 = 1.08+0.384 = 1.464$

Step 3: $w=1.464$, $dL/dw=-3.072$, $w_4 = 1.464+0.307 = 1.771$

Step 4: $w=1.771$, $dL/dw=-2.458$, $w_5 = 1.771+0.246 = 2.017$

Converging toward $w^*=3$ (true minimum). With each step loss decreases.

$L(w=0)=9$, $L(w=0.6)=5.76$, $L(w=1.08)=3.69$, $L(w=2.017)=0.963$ — shrinking!

COMMON MISTAKE / PITFALL

Effect of Learning Rate:

- eta too small (0.001): very slow convergence, may need thousands of steps.
- eta too large (>1.0): overshoots minimum, may diverge (loss grows).
- eta optimal: typically found by grid search or schedulers.

3.2 Backpropagation — Complete Worked Example

KEY CONCEPT

The Chain Rule: If L depends on a , a depends on z , z depends on w :

$$dL/dw = (dL/da) * (da/dz) * (dz/dw)$$

Algorithm: (1) Forward pass → compute all activations. (2) Compute loss. (3) Backward pass → propagate dL/dw for every weight.

SOLVED EXAMPLE

Problem: 1-hidden-layer network: $x=2.0$, $y_{\text{true}}=1.0$. $w_1=0.5$ (input→hidden), $w_2=0.8$ (hidden→output), $b_1=b_2=0$. Hidden: sigmoid; Output: sigmoid; Loss: MSE. Run one backprop step ($\eta=0.5$).

FORWARD PASS:

$$z_1 = w_1 * x + b_1 = 0.5 * 2.0 + 0 = 1.0$$

$$a_1 = \sigma(z_1) = \sigma(1.0) = 0.7311$$

$$z_2 = w_2 * a_1 + b_2 = 0.8 * 0.7311 + 0 = 0.5849$$

$$a_2 = \sigma(z_2) = \sigma(0.5849) = 0.6424$$

$$\text{Loss } L = (y_{\text{true}} - a_2)^2 = (1.0 - 0.6424)^2 = (0.3576)^2 = 0.1279$$

BACKWARD PASS — Output Layer:

$$dL/da_2 = -2 * (y_{\text{true}} - a_2) = -2 * 0.3576 = -0.7152$$

$$da_2/dz_2 = a_2 * (1 - a_2) = 0.6424 * (1 - 0.6424) = 0.6424 * 0.3576 = 0.2297$$

$$dL/dz_2 = dL/da_2 * da_2/dz_2 = -0.7152 * 0.2297 = -0.1643$$

$$dL/dw_2 = dL/dz_2 * dz_2/dw_2 = dL/dz_2 * a_1 = -0.1643 * 0.7311 = -0.1201$$

BACKWARD PASS — Hidden Layer:

$$dL/da_1 = dL/dz_2 * dz_2/da_1 = -0.1643 * w_2 = -0.1643 * 0.8 = -0.1314$$

$$da_1/dz_1 = a_1 * (1 - a_1) = 0.7311 * 0.2689 = 0.1966$$

$$dL/dz_1 = -0.1314 * 0.1966 = -0.02583$$

$$dL/dw_1 = dL/dz_1 * dz_1/dw_1 = -0.02583 * x = -0.02583 * 2.0 = -0.05166$$

WEIGHT UPDATE ($\eta=0.5$):

$$w_{2_new} = 0.8 - 0.5 * (-0.1201) = 0.8 + 0.0601 = 0.8601$$

$$w_{1_new} = 0.5 - 0.5 * (-0.05166) = 0.5 + 0.02583 = 0.5258$$

After update, run forward again: loss decreases from 0.1279 toward 0.

3.3 Computational Graphs & AutoGrad

KEY CONCEPT

A computational graph is a DAG (directed acyclic graph) where nodes are operations and edges carry values. AutoGrad traces this graph during the forward pass, then runs backprop in reverse.

Each node stores: (1) its forward output, (2) how to compute local gradient.

SOLVED EXAMPLE

Problem: $f(x,y) = (x+y) * \sin(x)$. Find df/dx and df/dy at $x=\pi/2$, $y=1$.

Computational graph nodes:

$$a = x + y$$

$$b = \sin(x)$$

$$f = a * b$$

Forward pass at $x=\pi/2 \approx 1.5708$, $y=1$:

$$a = 1.5708 + 1 = 2.5708$$

$$b = \sin(1.5708) = 1.0$$

$$f = 2.5708 * 1.0 = 2.5708$$

Backward pass ($df/df=1$):

$$df/da = b = 1.0$$

$$df/db = a = 2.5708$$

$$da/dx = 1, da/dy = 1$$

$$db/dx = \cos(x) = \cos(\pi/2) = 0.0$$

$$df/dx = df/da * da/dx + df/db * db/dx = 1.0 * 1 + 2.5708 * 0.0 = 1.0$$

$$df/dy = df/da * da/dy = 1.0 * 1 = 1.0$$

3.4 SGD Variants — Numerical Comparison

KEY CONCEPT

Vanilla SGD: $w = w - \eta * g$

SGD + Momentum: $v = \beta * v - \eta * g$; $w = w + v$

AdaGrad: $h = h + g^2$; $w = w - (\eta / \sqrt{h + \epsilon}) * g$

RMSprop: $h = \beta * h + (1 - \beta) * g^2$; $w = w - (\eta / \sqrt{h + \epsilon}) * g$

SOLVED EXAMPLE

Problem: $L=w^2$. $g=-2w$. Start $w=5.0$, $\eta=0.1$. Run 1 step for SGD, Momentum, AdaGrad.

Gradient at $w=5$: $g = -2 * 5 \dots$ wait, $L=w^2$, $dL/dw=2w=10$

Vanilla SGD:

$$w_{\text{new}} = 5.0 - 0.1 \cdot 10 = 4.0$$

SGD + Momentum (beta=0.9, v0=0):

$$v = 0.9 \cdot 0 - 0.1 \cdot 10 = -1.0$$

$$w_{\text{new}} = 5.0 + (-1.0) = 4.0 \text{ [same as SGD for first step]}$$

But in step 2: v carries memory $\rightarrow$ faster convergence

AdaGrad (h0=0, eps=1e-8):

$$h = 0 + 10^2 = 100$$

$$\text{effective_lr} = 0.1 / \sqrt{100 + 1e-8} = 0.1 / 10 = 0.01$$

$$w_{\text{new}} = 5.0 - 0.01 \cdot 10 = 4.9$$

AdaGrad is much more conservative (lr shrinks due to large gradient accumulation).

3.5 Adam Optimizer — Full Numerical Trace

KEY CONCEPT

Adam Algorithm:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \text{ [1st moment]}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \text{ [2nd moment]}$$

$$m_{\text{hat}} = m_t / (1 - \beta_1^t) \text{ [bias correction]}$$

$$v_{\text{hat}} = v_t / (1 - \beta_2^t) \text{ [bias correction]}$$

$$w_t = w_{t-1} - \eta \cdot m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon)$$

Defaults: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$, $\eta=0.001$

SOLVED EXAMPLE

Problem: $L=w^2$. $w_0=1.0$, $g_t=2w$. Run Adam for 3 steps. $\beta_1=0.9$, $\beta_2=0.999$, $\eta=0.1$, $\epsilon=1e-8$.

Initial: $m_0=0$, $v_0=0$

Step $t=1$, $w=1.0$, $g=2.0$:

$$m_1 = 0.9 \cdot 0 + 0.1 \cdot 2.0 = 0.2$$

$$v_1 = 0.999 \cdot 0 + 0.001 \cdot 4.0 = 0.004$$

$$m_{\text{hat}} = 0.2 / (1 - 0.9^1) = 0.2 / 0.1 = 2.0$$

$$v_{\text{hat}} = 0.004 / (1 - 0.999^1) = 0.004 / 0.001 = 4.0$$

$$w_1 = 1.0 - 0.1 \cdot 2.0 / (\sqrt{4.0} + 1e-8) = 1.0 - 0.2 / 2.0 = 1.0 - 0.1 = 0.9$$

Step t=2, w=0.9, g=1.8:

$$m_2 = 0.9 \cdot 0.2 + 0.1 \cdot 1.8 = 0.18 + 0.18 = 0.36$$

$$v_2 = 0.999 \cdot 0.004 + 0.001 \cdot 3.24 = 0.003996 + 0.00324 = 0.007236$$

$$m_{\text{hat}} = 0.36 / (1 - 0.81) = 0.36 / 0.19 = 1.895$$

$$v_{\text{hat}} = 0.007236 / (1 - 0.998) = 0.007236 / 0.002 = 3.618$$

$$w_2 = 0.9 - 0.1 \cdot 1.895 / (\sqrt{3.618} + 1e-8) = 0.9 - 0.1895 / 1.902 = 0.9 - 0.0997 = 0.800$$

Step t=3, w=0.800, g=1.6:

$$m_3 = 0.9 \cdot 0.36 + 0.1 \cdot 1.6 = 0.324 + 0.16 = 0.484$$

$$v_3 = 0.999 \cdot 0.007236 + 0.001 \cdot 2.56 = 0.007229 + 0.00256 = 0.009789$$

$$m_{\text{hat}} = 0.484 / (1 - 0.729) = 0.484 / 0.271 = 1.786$$

$$v_{\text{hat}} = 0.009789 / (1 - 0.997) = 0.009789 / 0.003 = 3.263$$

$$w_3 = 0.800 - 0.1 \cdot 1.786 / (\sqrt{3.263}) = 0.800 - 0.1786 / 1.807 = 0.800 - 0.0988 = 0.701$$

Sequence: $w=1.0 \rightarrow 0.9 \rightarrow 0.800 \rightarrow 0.701$, converging to $w^*=0$.

3.6 Regularization — Numerical Examples

KEY CONCEPT

L2 Regularization (Weight Decay):

$$L_{\text{total}} = L_{\text{original}} + (\lambda/2) \cdot ||w||^2$$

$$\text{Gradient: } dL_{\text{total}}/dw = dL/dw + \lambda \cdot w$$

$$\text{Update: } w = w - \eta \cdot (dL/dw + \lambda \cdot w) = (1 - \eta \cdot \lambda) \cdot w - \eta \cdot dL/dw$$

SOLVED EXAMPLE

Problem: $w=[2.0, -1.5]$, gradient from data: $g=[0.4, -0.6]$. $\eta=0.1$, $\lambda=0.01$. Compare update with and without L2.

Without L2:

$$w_{\text{new}} = [2.0 - 0.1 \cdot 0.4, -1.5 - 0.1 \cdot (-0.6)]$$

$$= [2.0 - 0.04, -1.5 + 0.06] = [1.96, -1.44]$$

With L2 ($\lambda=0.01$):

$$g_{\text{total}} = g + \lambda \cdot w = [0.4 + 0.01 \cdot 2.0, -0.6 + 0.01 \cdot (-1.5)]$$

$$= [0.4+0.02, -0.6-0.015] = [0.42, -0.615]$$

$$w_{\text{new}} = [2.0-0.1*0.42, -1.5-0.1*(-0.615)]$$

$$= [1.958, -1.4385]$$

L2 nudges weights slightly toward zero at each step (decay effect).

SOLVED EXAMPLE

Problem (Dropout): Layer output $a=[1.2, 0.8, -0.5, 2.1, 1.0]$. Dropout rate $p=0.5$ (keep each neuron with 50% probability). Mask = $[1,0,1,0,1]$. What is output during training and testing?

Training:

$$a_{\text{dropped}} = a * \text{mask} = [1.2*1, 0.8*0, -0.5*1, 2.1*0, 1.0*1]$$

$$= [1.2, 0.0, -0.5, 0.0, 1.0]$$

Testing (no dropout, scale by keep_prob=0.5):

$$a_{\text{test}} = a * 0.5 = [0.6, 0.4, -0.25, 1.05, 0.5]$$

(Inverted dropout implementation scales during training by $1/p$ instead)

3.7 Batch Normalization — Full Numerical Walkthrough

KEY CONCEPT

For a mini-batch $\{x_1, \dots, x_m\}$:

$$\mu = (1/m) * \text{SUM } x_i$$

$$\text{sigma}^2 = (1/m) * \text{SUM } (x_i - \mu)^2$$

$$x_{\text{hat}}_i = (x_i - \mu) / \text{sqrt}(\text{sigma}^2 + \text{eps})$$

$$y_i = \text{gamma} * x_{\text{hat}}_i + \text{beta} \text{ [gamma, beta are learned]}$$

SOLVED EXAMPLE

Problem: Mini-batch inputs (one feature): $x=[2, 4, 6, 8]$. Compute BN output with $\text{gamma}=1.5$, $\text{beta}=0.5$, $\text{eps}=1e-5$.

$$m = 4$$

$$\mu = (2+4+6+8)/4 = 20/4 = 5.0$$

$$\text{sigma}^2 = [(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2]/4$$

$$= [9+1+1+9]/4 = 20/4 = 5.0$$

$$\text{sigma} = \text{sqrt}(5.0+1e-5) \approx 2.2361$$

$$x_{\text{hat}} = [(2-5)/2.2361, (4-5)/2.2361, (6-5)/2.2361, (8-5)/2.2361]$$

$$= [-1.3416, -0.4472, 0.4472, 1.3416]$$


```

y = gamma*x_hat + beta = 1.5*x_hat + 0.5:

y = [1.5*(-1.3416)+0.5, 1.5*(-0.4472)+0.5, 1.5*0.4472+0.5,
     1.5*1.3416+0.5]

= [-1.5124, -0.1708, 1.1708, 2.5124]

```

Mean of y_{hat} is 0 (before scaling). γ & β give the network control to shift/scale the normalized distribution.

SOLVED EXAMPLE

Problem (Layer Normalization): Single sample, one layer with 4 neurons: $x=[3, 1, 4, 2]$. Compute LayerNorm output ($\gamma=1$, $\beta=0$, $\epsilon=1e-5$).

```

d = 4 (number of neurons in layer)

mu = (3+1+4+2)/4 = 10/4 = 2.5

sigma^2 = [(3-2.5)^2+(1-2.5)^2+(4-2.5)^2+(2-2.5)^2]/4
= [0.25+2.25+2.25+0.25]/4 = 5.0/4 = 1.25

sigma = sqrt(1.25) = 1.118

x_hat = [(3-2.5)/1.118, (1-2.5)/1.118, (4-2.5)/1.118, (2-2.5)/1.118]
= [0.4472, -1.3416, 1.3416, -0.4472]

y = 1*x_hat + 0 = [0.4472, -1.3416, 1.3416, -0.4472]

```

Layer Norm normalizes across features for each sample — unlike BN which normalizes across batch.

3.8 Weight Initialization — Theory & Numerics

KEY CONCEPT

Xavier (Glorot): For sigmoid/tanh. $W \sim N(0, 2/(n_{\text{in}}+n_{\text{out}}))$

He: For ReLU. $W \sim N(0, 2/n_{\text{in}})$ [accounts for half neurons zeroed by ReLU]

Why not all zeros? All neurons compute same gradient $\rightarrow$ same update $\rightarrow$ never differentiate.

SOLVED EXAMPLE

Problem: Layer has $n_{\text{in}}=256$, $n_{\text{out}}=128$. What are Xavier and He std deviations?

```

Xavier std = sqrt(2/(n_in + n_out)) = sqrt(2/384) = sqrt(0.00521) =
0.0721

```

```

He std = sqrt(2/n_in) = sqrt(2/256) = sqrt(0.00781) = 0.0884

```

He is slightly larger — compensates for ReLU killing ~50% of activations.

Convolutional Neural Networks

CNNs exploit local spatial structure. This chapter gives detailed convolution calculations, output size formulas, pooling numericals, and parameter count computations.

4.1 Convolution Operation — Step-by-Step

KEY CONCEPT

$$(I * K)[i,j] = \sum_m \sum_n I[i+m, j+n] * K[m,n]$$

Slide kernel K over input I. At each position, element-wise multiply and sum.

$$\text{Output size: } H_{\text{out}} = (H_{\text{in}} - F + 2P)/S + 1$$

$$W_{\text{out}} = (W_{\text{in}} - F + 2P)/S + 1$$

SOLVED EXAMPLE

Problem: Convolve input I (4×4) with kernel K (2×2), no padding, stride=1.

Input I: Kernel K:

```
1 2 3 4 1 0
5 6 7 8 0 -1
9 10 11 12
13 14 15 16
```

Output size:

$$H_{\text{out}} = (4-2+0)/1 + 1 = 3, W_{\text{out}} = 3 \rightarrow 3 \times 3 \text{ output}$$

Computing each output element (I*K)[i,j]:

```
[0,0]: 1*1 + 2*0 + 5*0 + 6*(-1) = 1+0+0-6 = -5
[0,1]: 2*1 + 3*0 + 6*0 + 7*(-1) = 2-7 = -5
[0,2]: 3*1 + 4*0 + 7*0 + 8*(-1) = 3-8 = -5
[1,0]: 5*1 + 6*0 + 9*0 + 10*(-1) = 5-10 = -5
[1,1]: 6*1 + 7*0 + 10*0 + 11*(-1) = 6-11 = -5
[1,2]: 7*1 + 8*0 + 11*0 + 12*(-1) = 7-12 = -5
[2,0]: 9*1 + 10*0 + 13*0 + 14*(-1) = 9-14 = -5
[2,1]: 10*1 + 11*0 + 14*0 + 15*(-1) = 10-15 = -5
[2,2]: 11*1 + 12*0 + 15*0 + 16*(-1) = 11-16 = -5
```

Output: $\begin{bmatrix} -5 & -5 & -5 \\ -5 & -5 & -5 \\ -5 & -5 & -5 \end{bmatrix}$

This kernel (top-left minus bottom-right) detects diagonal edges.

SOLVED EXAMPLE

Problem: Input 5x5, kernel 3x3. Compute output sizes for: (a) no padding, stride=1. (b) same padding, stride=1. (c) no padding, stride=2.

(a) $P=0, S=1: H_{out} = (5-3+0)/1+1 = 3 \rightarrow 3 \times 3$

(b) Same padding: $P = (F-1)/2 = 1$

$H_{out} = (5-3+2)/1+1 = 5 \rightarrow 5 \times 5$ (same as input)

(c) $P=0, S=2: H_{out} = (5-3+0)/2+1 = 2 \rightarrow 2 \times 2$

4.2 Multi-Channel Convolution & Parameter Count

KEY CONCEPT

Input: $H \times W \times C_{in}$ (e.g. RGB image: $224 \times 224 \times 3$)

Filter bank: $F \times F \times C_{in} \times C_{out}$ (C_{out} kernels, each $F \times F \times C_{in}$)

Params in conv layer = $F \times F \times C_{in} \times C_{out} + C_{out}$ [+ C_{out} biases]

Output: $H_{out} \times W_{out} \times C_{out}$

SOLVED EXAMPLE

Problem: AlexNet first layer: input $227 \times 227 \times 3$, filter 11×11 , stride=4, padding=0, $C_{out}=96$. Find (a) output size, (b) number of parameters.

(a) $H_{out} = (227-11+0)/4 + 1 = 216/4 + 1 = 54+1 = 55$

Output: $55 \times 55 \times 96$

(b) Params = $11 \times 11 \times 3 \times 96 + 96 = 34848 + 96 = 34,944$

Just the first conv layer has 34,944 parameters!

SOLVED EXAMPLE

Problem (Parameter counting for a full CNN):

Architecture: Input $32 \times 32 \times 1 \rightarrow \text{Conv}(3 \times 3, 16) \rightarrow \text{Conv}(3 \times 3, 32) \rightarrow \text{FC}(128) \rightarrow \text{FC}(10)$

All convolutions: padding=1, stride=1.

Layer 1 (Conv $3 \times 3 \times 1 \rightarrow 16$): params = $3 \times 3 \times 1 \times 16 + 16 = 144 + 16 = 160$

Output: $32 \times 32 \times 16$ (same padding maintains spatial size)

Layer 2 (Conv $3 \times 3 \times 16 \rightarrow 32$): params = $3 \times 3 \times 16 \times 32 + 32 = 4608 + 32 = 4,640$

Output: $32 \times 32 \times 32$

Flatten: $32 \times 32 \times 32 = 32,768$ neurons

FC1 ($32768 \rightarrow 128$): $\text{params} = 32768 \times 128 + 128 = 4,194,304 + 128 = 4,194,432$

FC2 ($128 \rightarrow 10$): $\text{params} = 128 \times 10 + 10 = 1280 + 10 = 1,290$

TOTAL: $160 + 4,640 + 4,194,432 + 1,290 = 4,200,522$ parameters

Notice: FC layers dominate parameters! That is why GAP is used in modern CNNs.

4.3 Pooling — Numericals

SOLVED EXAMPLE

Problem: Apply 2×2 max-pooling (stride=2) to input:

Input :

3 8 1 4

7 2 5 6

1 9 3 2

4 5 8 1

Top-left 2×2 region:

$[[3, 8], [7, 2]] \rightarrow \text{max} = 8$

Top-right 2×2 region:

$[[1, 4], [5, 6]] \rightarrow \text{max} = 6$

Bottom-left 2×2 region:

$[[1, 9], [4, 5]] \rightarrow \text{max} = 9$

Bottom-right 2×2 region:

$[[3, 2], [8, 1]] \rightarrow \text{max} = 8$

Max-Pool Output: $[[8, 6],$

$[9, 8]]$

Average-Pool Output:

Top-left: $(3+8+7+2)/4=5.0$, Top-right: $(1+4+5+6)/4=4.0$

Bot-left: $(1+9+4+5)/4=4.75$, Bot-right: $(3+2+8+1)/4=3.5$

Avg-Pool: $[[5.0, 4.0], [4.75, 3.5]]$

4.4 CNN Architectures — Key Numbers to Know

Architecture	Year	Input	Depth	Params	Key Innovation
AlexNet	2012	227x227x3	8 layers	62M	ReLU+Dropout+GPU, large kernels (11x11)
VGGNet-16	2014	224x224x3	16 layers	138M	Only 3x3 kernels, very deep
GoogLeNet	2014	224x224x3	22 layers	6.8M	Inception block (parallel kernels), 1x1 conv
ResNet-50	2016	224x224x3	50 layers	25M	Skip connections, Batch Norm, bottleneck
U-Net	2015	572x572x1	23 layers	31M	Encoder-Decoder + skip connections (segmentation)

SOLVED EXAMPLE

Problem (ResNet Skip Connection): Layer input $x=[1.0, -0.5, 2.0]$. $F(x) = \text{ReLU}(Wx+b)$. $W=[[0.5,0,0],[0,0.5,0],[0,0,0.5]]$, $b=[0,0,0]$. Compute $H(x)=F(x)+x$ (ResNet residual block).

$$Wx+b = [0.5*1.0, 0.5*(-0.5), 0.5*2.0] = [0.5, -0.25, 1.0]$$

$$F(x) = \text{ReLU}([0.5, -0.25, 1.0]) = [0.5, 0.0, 1.0]$$

$$H(x) = F(x) + x = [0.5+1.0, 0.0+(-0.5), 1.0+2.0] = [1.5, -0.5, 3.0]$$

Note: even though second neuron has $F(x)=0$ (killed by ReLU), the skip adds x back: $0+(-0.5)=-0.5$. Gradient can flow through skip!

Recurrent Neural Networks

RNNs process sequences by maintaining hidden state. We cover vanilla RNN forward/backward, vanishing gradient analysis, and full LSTM/GRU numerical traces.

5.1 Vanilla RNN — Forward Pass Numerical

KEY CONCEPT

$$h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b)$$

$$y_t = W_y * h_t + b_y$$

All weight matrices are shared across all time steps!

SOLVED EXAMPLE

Problem: 1D RNN. $W_h=0.8$, $W_x=0.5$, $b=0$. Input sequence: $x=[1.0, 0.5, -1.0]$. $h_0=0$. Compute h_1, h_2, h_3 .

$$\begin{aligned} h_1 &= \tanh(0.8 * h_0 + 0.5 * x_1 + 0) \\ &= \tanh(0.8 * 0 + 0.5 * 1.0) = \tanh(0.5) = 0.4621 \end{aligned}$$

$$\begin{aligned} h_2 &= \tanh(0.8 * h_1 + 0.5 * x_2) \\ &= \tanh(0.8 * 0.4621 + 0.5 * 0.5) \\ &= \tanh(0.3697 + 0.25) = \tanh(0.6197) = 0.5527 \end{aligned}$$

$$\begin{aligned} h_3 &= \tanh(0.8 * h_2 + 0.5 * x_3) \\ &= \tanh(0.8 * 0.5527 + 0.5 * (-1.0)) \\ &= \tanh(0.4422 - 0.5) = \tanh(-0.0578) = -0.0577 \end{aligned}$$

Hidden state carries "memory" — h_3 is influenced by all past inputs.

5.2 Vanishing Gradient — Quantitative Analysis

KEY CONCEPT

Gradient of loss at time T w.r.t. h_t involves product of $T-t$ Jacobians:

$$dL/dh_t = (dh_T/dh_t) * dL/dh_T$$

$$dh_T/dh_t = \text{PRODUCT}_{k=t+1}^{T} (dh_k/dh_{k-1})$$

Each factor $dh_k/dh_{k-1} = \text{diag}(1 - \tanh^2(z_k)) * W_h$

If |eigenvalues of W_h | < 1: gradient $\rightarrow 0$ exponentially. If > 1: explodes.

SOLVED EXAMPLE

Problem: $W_h=0.5$ (scalar RNN). Compute gradient scaling after $T=10$ steps.

Each step multiplies gradient by $W_h * \tanh_derivative \approx W_h * 1$ (for small tanh input)

After $T=10$ steps: $scaling = W_h^T = 0.5^{10} = 0.000977 \approx 0.001$

After $T=20$ steps: $0.5^{20} = 9.5e-7 \approx 0$

Gradient from 20 steps ago has essentially vanished — long-term memory lost!

Exploding case: $W_h=1.5$

After $T=10$ steps: $1.5^{10} = 57.7$

After $T=20$ steps: $1.5^{20} = 3325 \rightarrow$ gradient explodes!

Fix for exploding: Gradient clipping: if $\|g\| > threshold$, $g = g * threshold / \|g\|$

5.3 LSTM — Complete Numerical Trace

KEY CONCEPT

LSTM Equations (all σ = sigmoid):

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \text{ [forget gate]}$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i) \text{ [input gate]}$$

$$\tilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C) \text{ [candidate]}$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \text{ [cell state]}$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \text{ [output gate]}$$

$$h_t = o_t * \tanh(C_t) \text{ [hidden state]}$$

SOLVED EXAMPLE

Problem (Scalar LSTM): All weights=0.5, all biases=0. $h_0=0$, $C_0=0$. $x_1=1.0$. Compute h_1 , C_1 .

Concatenated input $[h_0, x_1] = [0, 1.0]$

(Using $W*[h, x] = W_h*h + W_x*x = 0.5*0 + 0.5*1.0 = 0.5$ for all gates)

$$f_1 = \sigma(0.5) = 1/(1+e^{-0.5}) = 0.6225$$

$$i_1 = \sigma(0.5) = 0.6225$$

$$\tilde{C}_1 = \tanh(0.5) = 0.4621$$

$$C_1 = f_1 * C_0 + i_1 * \tilde{C}_1 = 0.6225 * 0 + 0.6225 * 0.4621$$

$$= 0.2877$$

$$o_1 = \sigma(0.5) = 0.6225$$

$$h_1 = o_1 * \tanh(C_1) = 0.6225 * \tanh(0.2877)$$

$$= 0.6225 * 0.2810 = 0.1750$$

Step t=2: x2=0.0 (silence):

$$\text{Input } [h_1, x_2] = [0.175, 0.0]$$

$$W*[h, x] = 0.5*0.175 + 0.5*0.0 = 0.0875$$

$$f_2 = \sigma(0.0875) = 0.5219$$

$$i_2 = \sigma(0.0875) = 0.5219$$

$$C\sim_2 = \tanh(0.0875) = 0.0874$$

$$C_2 = f_2*C_1 + i_2*C\sim_2 = 0.5219*0.2877 + 0.5219*0.0874$$

$$= 0.1502 + 0.0456 = 0.1958$$

$$o_2 = \sigma(0.0875) = 0.5219$$

$$h_2 = 0.5219 * \tanh(0.1958) = 0.5219 * 0.1943 = 0.1014$$

Cell state C retains ~52% of its value (forget gate ~0.52) even with zero input.

5.4 GRU — Numerical Example

KEY CONCEPT

$$r_t = \sigma(W_r [h_{t-1}, x_t]) \text{ [reset gate]}$$

$$u_t = \sigma(W_u [h_{t-1}, x_t]) \text{ [update gate]}$$

$$h\sim_t = \tanh(W [r_t*h_{t-1}, x_t]) \text{ [candidate hidden]}$$

$$h_t = u_t*h_{t-1} + (1-u_t)*h\sim_t \text{ [new hidden state]}$$

SOLVED EXAMPLE

Problem: Scalar GRU. $W_r=W_u=W=0.5$, $b=0$. $h_0=0.4$, $x_1=1.0$.

$$[h_0, x_1] = [0.4, 1.0]$$

$$W*[h, x] = 0.5*0.4 + 0.5*1.0 = 0.7$$

$$r_1 = \sigma(0.7) = 0.6682$$

$$u_1 = \sigma(0.7) = 0.6682$$

$$r_1 * h_0 = 0.6682 * 0.4 = 0.2673$$

$$h\sim_1 = \tanh(W * [r_1*h_0, x_1]) = \tanh(0.5*0.2673 + 0.5*1.0)$$

$$= \tanh(0.1337 + 0.5) = \tanh(0.6337) = 0.5584$$

$$h_{_1} = u_{_1} \cdot h_0 + (1 - u_{_1}) \cdot \tilde{h}_{_1}$$

$$= 0.6682 \cdot 0.4 + 0.3318 \cdot 0.5584$$

$$= 0.2673 + 0.1852 = 0.4525$$

GRU update gate $u=0.67 \rightarrow$ keeps 67% of old state, 33% of new candidate.

5.5 Backpropagation Through Time (BPTT)

SOLVED EXAMPLE

Problem: 2-step RNN. $x=[1,0]$, $y_{\text{true}}=[0,1]$. $W_h=0.5$, $W_x=1.0$, $W_y=1.0$. $h_0=0$. Loss = MSE on y_2 only. Compute dL/dW_x via BPTT.

Forward:

$$h_1 = \tanh(0.5 \cdot 0 + 1.0 \cdot 1) = \tanh(1.0) = 0.7616$$

$$h_2 = \tanh(0.5 \cdot 0.7616 + 1.0 \cdot 0) = \tanh(0.3808) = 0.3643$$

$$y_2_{\text{hat}} = 1.0 \cdot 0.3643 = 0.3643$$

$$L = (1 - 0.3643)^2 = (0.6357)^2 = 0.4041$$

Backward (only through $t=2$ and $t=1$):

$$dL/dy_2_{\text{hat}} = -2 \cdot (1 - 0.3643) = -1.2714$$

$$dL/dh_2 = dL/dy_2_{\text{hat}} \cdot W_y = -1.2714 \cdot 1.0 = -1.2714$$

$$dh_2/dz_2 = 1 - \tanh^2(z_2) = 1 - 0.3643^2 = 1 - 0.1327 = 0.8673$$

$$dL/dz_2 = -1.2714 \cdot 0.8673 = -1.1030$$

$$dL/dW_x \text{ (from } t=2) = dL/dz_2 \cdot x_2 = -1.1030 \cdot 0 = 0$$

$$dL/dh_1 = dL/dz_2 \cdot W_h = -1.1030 \cdot 0.5 = -0.5515$$

$$dh_1/dz_1 = 1 - 0.7616^2 = 1 - 0.5800 = 0.4200$$

$$dL/dz_1 = -0.5515 \cdot 0.4200 = -0.2316$$

$$dL/dW_x \text{ (from } t=1) = dL/dz_1 \cdot x_1 = -0.2316 \cdot 1.0 = -0.2316$$

$$\text{Total } dL/dW_x = 0 + (-0.2316) = -0.2316$$

Attention Mechanism & Transformers

Attention is the core of modern NLP. We work through QKV computation, softmax attention, multi-head attention, and positional encoding — all with full numerical examples.

6.1 Scaled Dot-Product Attention — Numerical

KEY CONCEPT

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) * V$$

d_k : dimension of key vectors. Scaling prevents dot products from becoming too large (which would cause softmax gradients to vanish).

SOLVED EXAMPLE

Problem: 3 tokens, $d_k=2$, $d_v=2$.

$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ (3 queries)

$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ (3 keys)

$V = \begin{bmatrix} 10 & 0 \\ 0 & 10 \\ 5 & 5 \end{bmatrix}$ (3 values)

Step 1: Compute $Q K^T$ (3x3 matrix):

$$Q K^T [0,0] = [1,0] \cdot [1,0] = 1$$

$$Q K^T [0,1] = [1,0] \cdot [0,1] = 0$$

$$Q K^T [0,2] = [1,0] \cdot [1,1] = 1$$

$$Q K^T [1,0] = [0,1] \cdot [1,0] = 0$$

$$Q K^T [1,1] = [0,1] \cdot [0,1] = 1$$

$$Q K^T [1,2] = [0,1] \cdot [1,1] = 1$$

$$Q K^T [2,0] = [1,1] \cdot [1,0] = 1$$

$$Q K^T [2,1] = [1,1] \cdot [0,1] = 1$$

$$Q K^T [2,2] = [1,1] \cdot [1,1] = 2$$

$$Q K^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

Step 2: Scale by $1/\sqrt{d_k} = 1/\sqrt{2} = 0.707$:

$$\text{Scores} = 0.707 * \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0.707 & 0 & 0.707 \\ 0 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{bmatrix}$$

Step 3: Apply softmax to each row:

Row 0: $\exp([0.707, 0, 0.707]) = [2.028, 1.0, 2.028]$, $\text{sum}=5.056$

$\text{softmax} = [0.401, 0.198, 0.401]$

Row 1: $\exp([0, 0.707, 0.707]) = [1.0, 2.028, 2.028]$, $\text{sum}=5.056$

$\text{softmax} = [0.198, 0.401, 0.401]$

Row 2: $\exp([0.707, 0.707, 1.414]) = [2.028, 2.028, 4.112]$, $\text{sum}=8.168$

$\text{softmax} = [0.248, 0.248, 0.503]$

Step 4: Multiply attention weights by V:

$\text{Attn} = [[0.401, 0.198, 0.401], [0.198, 0.401, 0.401], [0.248, 0.248, 0.503]]$

Output row 0 = $0.401 \cdot [10, 0] + 0.198 \cdot [0, 10] + 0.401 \cdot [5, 5]$

= $[4.01, 0] + [0, 1.98] + [2.005, 2.005]$

= $[6.015, 3.985]$

Output row 1 = $0.198 \cdot [10, 0] + 0.401 \cdot [0, 10] + 0.401 \cdot [5, 5]$

= $[1.98, 0] + [0, 4.01] + [2.005, 2.005]$

= $[3.985, 6.015]$

Output row 2 = $0.248 \cdot [10, 0] + 0.248 \cdot [0, 10] + 0.503 \cdot [5, 5]$

= $[2.48, 0] + [0, 2.48] + [2.515, 2.515]$

= $[4.995, 4.995]$

Token 2 (query=[1,1]) attends most to token 2 (key=[1,1], highest score 2).

6.2 Multi-Head Attention — Concept & Dimensions

KEY CONCEPT

$\text{head}_i = \text{Attention}(Q W^Q_i, K W^K_i, V W^V_i)$

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$

With h heads and d_{model} dimensions: each head has $d_k = d_{\text{model}}/h$.

Parameters per layer: $4 * d_{\text{model}}^2$ (W_Q, W_K, W_V, W_O each $d_{\text{model}} \times d_{\text{model}}$).

SOLVED EXAMPLE

Problem: Transformer with $d_{\text{model}}=512$, $h=8$ heads. Find d_k , d_v , and total parameters for one multi-head attention layer.

$$d_k = d_v = d_{\text{model}}/h = 512/8 = 64$$

Per head: W^Q_i (512x64), W^K_i (512x64), W^V_i (512x64)

Total for Q,K,V across all heads: $3 * 512 * 64 * 8 = 786,432$

W^O (concatenated heads $\rightarrow d_{\text{model}}$): $512 * 512 = 262,144$

Total attention params = $786432 + 262144 = 1,048,576 \approx 1\text{M}$ params per layer

GPT-3 (96 layers, $d_{\text{model}}=12288$, 96 heads): ~ 3.7 billion parameters total!

6.3 Positional Encoding — Numerical

KEY CONCEPT

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{\{2i/d_{\text{model}}\}})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{\{2i/d_{\text{model}}\}})$$

Added to word embeddings so the model knows token positions.

SOLVED EXAMPLE

Problem: $d_{\text{model}}=4$. Compute PE for positions $\text{pos}=0$ and $\text{pos}=1$.

For $\text{pos}=0$, $i=0$: $\text{PE}(0,0)=\sin(0/10000^0)=\sin(0)=0.0$

$\text{PE}(0,1)=\cos(0/10000^0)=\cos(0)=1.0$

$i=1$: $\text{PE}(0,2)=\sin(0/10000^{\{2/4\}})=\sin(0)=0.0$

$\text{PE}(0,3)=\cos(0/10000^{\{2/4\}})=\cos(0)=1.0$

$\text{PE}(\text{pos}=0) = [0.0, 1.0, 0.0, 1.0]$

For $\text{pos}=1$, $i=0$: $\text{PE}(1,0)=\sin(1/1)=\sin(1.0)=0.8415$

$\text{PE}(1,1)=\cos(1/1)=\cos(1.0)=0.5403$

$i=1$: $\text{PE}(1,2)=\sin(1/10000^{\{0.5\}})=\sin(1/100)=\sin(0.01)=0.0100$

$\text{PE}(1,3)=\cos(0.01)=0.9999$

$\text{PE}(\text{pos}=1) = [0.8415, 0.5403, 0.0100, 0.9999]$

Different positions $\rightarrow$ different PE vectors. Model can distinguish order.

Autoencoders

Autoencoders learn compact latent representations. We cover the basic AE, denoising, contractive, and the VAE with its reparameterization trick — all with full numericals.

7.1 Basic Autoencoder — Forward & Loss

KEY CONCEPT

Encoder: $z = f(x) = \text{activation}(W_e x + b_e)$

Decoder: $x' = g(z) = \text{activation}(W_d z + b_d)$

$$\text{Reconstruction Loss} = (1/N) * \text{SUM } ||x_i - x'_i||^2$$

SOLVED EXAMPLE

Problem: 3D input $x=[2, 1, 3]$. Encoder: $W_e=[[1,0,0],[0,0,1]]$, $b_e=[0,0]$, sigmoid. Decoder: $W_d=W_e^T$, $b_d=[0,0,0]$, linear. Compute x' and MSE loss.

Encoder (3→2):

$$z_{\text{linear}} = W_e * x = [[1,0,0],[0,0,1]] * [2,1,3] = [2, 3]$$

$$z = \text{sigmoid}([2, 3]) = [1/(1+e^{-2}), 1/(1+e^{-3})]$$

$$= [0.8808, 0.9526]$$

Decoder (2→3):

$$W_d = W_e^T = [[1,0],[0,0],[0,1]]$$

$$x' = W_d * z = [[1,0],[0,0],[0,1]] * [0.8808, 0.9526]$$

$$= [0.8808, 0.0, 0.9526]$$

MSE Loss:

$$L = ||(x - x')||^2 / 3$$

$$= [(2-0.8808)^2 + (1-0)^2 + (3-0.9526)^2] / 3$$

$$= [1.2530 + 1.0 + 4.1928] / 3$$

$$= 6.4458 / 3 = 2.149$$

High loss — encoder discards $x_2=1$ (middle feature). After training, encoder would learn to encode more representative features.

7.2 Denoising Autoencoder — Numerical

KEY CONCEPT

Train with corrupted input x_{noisy} , but measure reconstruction against clean x .

$$x_{\text{noisy}} = x + \text{noise}, \text{ noise} \sim N(0, \sigma^2 I)$$

$$\text{Loss} = ||x - \text{decoder}(\text{encoder}(x_{\text{noisy}}))||^2$$

Forces encoder to learn features robust to noise (cannot just copy input).

SOLVED EXAMPLE

Problem: Clean $x=[1.0, 2.0, 0.5]$. Add Gaussian noise $\sigma=0.2$. Compute corrupted input and note the loss structure.

$\text{noise} \sim N(0, 0.04)$: say $\text{noise}=[0.15, -0.18, 0.23]$ (sampled values)

$$x_{\text{tilde}} = x + \text{noise} = [1.15, 1.82, 0.73]$$

Encoder processes $x_{\text{tilde}} \rightarrow z_{\text{noisy}} \rightarrow \text{decoder} \rightarrow x_{\text{reconstructed}}$

$$\text{Loss} = ||x_{\text{clean}} - x_{\text{reconstructed}}||^2 \text{ (NOT } ||x_{\text{tilde}} - x_{\text{reconstructed}}||^2)$$

Why better than standard AE?

Standard AE could learn identity (copy input). Adding noise forces it to learn the true underlying structure, not just memorize.

7.3 Variational Autoencoder (VAE) — Complete Numerical

KEY CONCEPT

Encoder outputs: μ (mean) and $\log_var$ (log variance) of latent distribution.

Reparameterization: $z = \mu + \epsilon * \sigma$, where $\epsilon \sim N(0, I)$

$$\text{ELBO Loss} = \text{Reconstruction_Loss} + \text{KL_Loss}$$

$$\text{KL_Loss} = (1/2) * \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log(\sigma_j^2))$$

SOLVED EXAMPLE

Problem: 2D latent space. Encoder outputs $\mu=[-0.5, 0.3]$, $\log_var=[-0.4, 0.2]$. $\epsilon=[0.8, -0.6]$. Compute z , KL loss. Reconstruction loss = 1.5.

Step 1: Compute σ from $\log_var$:

$$\sigma_1 = \exp(\log_var_1 / 2) = \exp(-0.4/2) = \exp(-0.2) = 0.8187$$

$$\sigma_2 = \exp(\log_var_2 / 2) = \exp(0.2/2) = \exp(0.1) = 1.1052$$

Step 2: Sample z (reparameterization trick):

$$z_1 = \mu_1 + \sigma_1 * \epsilon_1 = -0.5 + 0.8187 * 0.8 = -0.5 + 0.6550 = 0.155$$

$$z_2 = \mu_2 + \sigma_2 * \epsilon_2 = 0.3 + 1.1052 * (-0.6) = 0.3 - 0.6631 = -0.363$$

$$z = [0.155, -0.363]$$

Step 3: Compute KL divergence:

$$KL = 0.5 * \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log_{\text{var}}_j)$$

$$\text{For } j=1: \sigma^2=e^{-0.4}=0.6703, \mu^2=0.25, \log_{\text{var}}=-0.4$$

$$\text{term1} = 0.5 * (0.6703 + 0.25 - 1 - (-0.4)) = 0.5 * (0.3203) = 0.1602$$

$$\text{For } j=2: \sigma^2=e^{0.2}=1.2214, \mu^2=0.09, \log_{\text{var}}=0.2$$

$$\text{term2} = 0.5 * (1.2214 + 0.09 - 1 - 0.2) = 0.5 * (0.1114) = 0.0557$$

$$KL_{\text{Loss}} = 0.1602 + 0.0557 = 0.2159$$

Step 4: Total ELBO Loss:

$$\text{Total} = \text{Reconstruction} + KL = 1.5 + 0.2159 = 1.7159$$

Generation: Sample $\epsilon \sim N(0, I)$, $z = \mu + \sigma * \epsilon$, decode $z \rightarrow$ new sample.

EXAM TIP

Key VAE Exam Points:

- The reparameterization trick makes sampling differentiable (gradients flow through z).
- KL term forces latent space to be close to $N(0, I) \rightarrow$ smooth interpolation possible.
- Beta-VAE: multiply KL by $\beta > 1 \rightarrow$ more disentangled representations.
- Standard AE cannot generate new data (latent space has "holes"). VAE can.

State-of-the-Art Models

This chapter covers Graph Neural Networks, Physics-Informed NNs, Kolmogorov-Arnold Networks, and Diffusion/Generative models — with numerical examples for each.

8.1 Graph Neural Networks — Message Passing Numerical

KEY CONCEPT

Message Passing Framework:

$$m_v^{(k)} = \text{AGGREGATE}(\{ h_u^{(k-1)} : u \in N(v) \})$$

$$h_v^{(k)} = \text{UPDATE}(h_v^{(k-1)}, m_v^{(k)})$$

GCN (simplest):

$$H^{(k)} = \text{sigma}(D^{-1/2} A D^{-1/2} H^{(k-1)} W^{(k)})$$

A = adjacency matrix (with self-loops), D = degree matrix, W = learnable weights.

SOLVED EXAMPLE

Problem: 3-node graph. Edges: 1-2, 2-3. Node features: $h_1=[1,0]$, $h_2=[0,1]$, $h_3=[1,1]$. $W=I$ (identity). Compute one GCN layer output.

Adjacency matrix A with self-loops:

A = $\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ (node 1 connects to self and node 2)

$\begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$ (node 2 connects to all)

$\begin{bmatrix} 0 & 1 & 1 \end{bmatrix}$ (node 3 connects to self and node 2)

Degree matrix D:

D = $\text{diag}(2, 3, 2)$ (row sums of A)

$$D^{-1/2} = \text{diag}(1/\sqrt{2}, 1/\sqrt{3}, 1/\sqrt{2})$$

$$= \text{diag}(0.7071, 0.5774, 0.7071)$$

Normalized $A_{\text{hat}} = D^{-1/2} A D^{-1/2}$:

$$A_{\text{hat}}[1,1] = D^{-1/2}[1] * A[1,1] * D^{-1/2}[1] = 0.7071 * 1 * 0.7071 = 0.500$$

$$A_{\text{hat}}[1,2] = 0.7071 * 1 * 0.5774 = 0.408$$

$$A_{\text{hat}}[1,3] = 0.7071 * 0 * 0.7071 = 0$$

$$A_{\text{hat}}[2,1] = 0.5774 * 1 * 0.7071 = 0.408$$

$$A_{\text{hat}}[2,2] = 0.5774 * 1 * 0.5774 = 0.333$$

$$A_hat[2,3] = 0.5774*1*0.7071 = 0.408$$

$$A_hat[3,2] = 0.7071*1*0.5774 = 0.408$$

$$A_hat[3,3] = 0.7071*1*0.7071 = 0.500$$

Multiply $A_hat * H$ ($H = \begin{bmatrix} 1,0 \\ 0,1 \\ 1,1 \end{bmatrix}$):

$$\text{Node 1: } 0.500*[1,0] + 0.408*[0,1] + 0*[1,1] = [0.500, 0.408]$$

$$\text{Node 2: } 0.408*[1,0] + 0.333*[0,1] + 0.408*[1,1] = [0.816, 0.741]$$

$$\text{Node 3: } 0*[1,0] + 0.408*[0,1] + 0.500*[1,1] = [0.500, 0.908]$$

After ReLU (all positive): same values.

Node 2 aggregates from all 3 neighbors → gets richer representation.

8.2 Physics-Informed Neural Networks (PINNs)

KEY CONCEPT

Key Idea: Add PDE residual as a loss term.

$$L = L_data + \lambda * L_physics$$

$$L_physics = (1/N_f) * \sum ||N[u(x_i, t_i); \theta] - f(x_i, t_i)||^2$$

Example: Heat equation: $du/dt = \alpha * d^2u/dx^2$. $N[u] = du/dt - \alpha*d^2u/dx^2$. At solution: $N[u]=0$.

SOLVED EXAMPLE

Problem: PINN for ODE: $du/dt = -2u$, $u(0)=1$. True solution: $u(t)=e^{-2t}$.

NN predicts $u(t)$. At collocation point $t=0.5$:

$$\text{True: } u(0.5) = e^{-1} = 0.3679$$

$$\text{NN prediction: } u_hat(0.5) = 0.42 \text{ [not yet converged]}$$

du_hat/dt at $t=0.5$ (via autodiff): -0.65 [computed by backprop through NN]

$$\text{PDE residual: } du_hat/dt + 2*u_hat = -0.65 + 2*0.42 = -0.65+0.84 = 0.19$$

$$L_physics = (0.19)^2 = 0.036$$

$$\text{If also have data point } (t=0, u=1): L_data = (1.0 - u_hat(0))^2$$

$$\text{Suppose } u_hat(0) = 0.98: L_data = 0.0004$$

$$\text{Total } L = 0.0004 + 1.0*0.036 = 0.0364$$

Training minimizes both: NN learns to satisfy the ODE everywhere, not just at data points.

8.3 Diffusion Models — Forward Process Numerical

KEY CONCEPT

Forward (adding noise): $q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t} \epsilon$$

$$\alpha_t = 1 - \beta_t, \alpha_{\bar{t}} = \prod_{s=1}^t \alpha_s$$

Reverse: NN learns to predict epsilon (noise) → subtract it to denoise.

SOLVED EXAMPLE

Problem (Scalar): $x_0=3.0$. $\beta_1=0.1$, $\beta_2=0.2$, $\beta_3=0.3$. Simulate forward process for $T=3$ steps.

$$\alpha_1=0.9, \alpha_2=0.8, \alpha_3=0.7$$

$$\alpha_{\bar{1}}=0.9, \alpha_{\bar{2}}=0.9*0.8=0.72, \\ \alpha_{\bar{3}}=0.72*0.7=0.504$$

$$t=1: x_1 = \sqrt{0.9}*3.0 + \sqrt{0.1}*\epsilon_1$$

$$= 0.9487*3.0 + 0.3162*\epsilon_1 = 2.846 + 0.316*\epsilon_1$$

$$\text{If } \epsilon_1=0.5: x_1 = 2.846+0.158 = 3.004 \text{ (small noise)}$$

$$t=2: x_2 = \sqrt{0.72}*3.0 + \sqrt{0.28}*\epsilon_2$$

$$= 0.8485*3.0 + 0.5292*\epsilon_2 = 2.546 + 0.529*\epsilon_2$$

$$\text{If } \epsilon_2=1.0: x_2 = 2.546+0.529 = 3.075$$

$$t=3: x_3 = \sqrt{0.504}*3.0 + \sqrt{0.496}*\epsilon_3$$

$$= 0.7099*3.0 + 0.7043*\epsilon_3 = 2.130 + 0.704*\epsilon_3$$

$$\text{If } \epsilon_3=-1.5: x_3 = 2.130-1.056 = 1.074$$

As t increases, signal x_0 weakens and noise increases. At $T \rightarrow 1000$: pure noise.

SOLVED EXAMPLE

Reverse Process (Denoising Step):

NN predicts noise $\epsilon_\theta(x_t, t)$. Then:

$$x_{t-1} = \left(\frac{1}{\sqrt{\alpha_t}} \right) * \left(x_t - \frac{(1-\alpha_t)}{\sqrt{1-\alpha_{\bar{t}}}} * \epsilon_\theta \right)$$

Example: $x_3=1.074$, $\alpha_3=0.7$, $\alpha_{\bar{3}}=0.504$. NN predicts $\epsilon=0.8$.

$$x_2 = \left(\frac{1}{\sqrt{0.7}} \right) * \left(1.074 - \frac{(0.3)}{\sqrt{0.496}} * 0.8 \right)$$

$$\begin{aligned}
 &= 1.1952 * (1.074 - 0.3/0.7043 * 0.8) \\
 &= 1.1952 * (1.074 - 0.3408) \\
 &= 1.1952 * 0.7332 = 0.876
 \end{aligned}$$

Each reverse step partially denoises. After T steps, get clean sample from noise.

8.4 Kolmogorov-Arnold Networks (KAN)

KEY CONCEPT

Kolmogorov-Arnold Theorem: Any multivariate continuous $f(x_1, \dots, x_n)$ can be written as:

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

KAN implements this: learnable 1D functions (splines) on *edges* instead of fixed activations on nodes.

Advantage: More interpretable — each edge function can be visualized as a 1D curve.

SOLVED EXAMPLE

Problem: 2-input KAN approximating $f(x_1, x_2) = x_1^2 + x_2^2$ (toy example). Inner functions $\phi_{1,1}(x_1) = x_1^2$, $\phi_{1,2}(x_2) = x_2^2$. Outer function $\Phi_1(z) = z$. Evaluate at $x_1=2$, $x_2=3$.

$$\text{Inner: } \phi_{1,1}(2) = 4, \quad \phi_{1,2}(3) = 9$$

$$\text{Sum: } z = 4 + 9 = 13$$

$$\text{Outer: } \Phi_1(13) = 13$$

$$f(2, 3) = 13 \quad \checkmark \quad (\text{true: } 4+9=13)$$

Unlike MLP which uses fixed ReLU/sigmoid, KAN learns the " x^2 " shape on each edge.

PRACTICE PROBLEMS

Try these problems yourself before checking answers. They are exam-style questions.

PRACTICE PROBLEM

Problem 1 (Ch 1): Compute $\text{sigmoid}(3)$ and its gradient. Then compute binary cross-entropy if $y_{\text{true}}=1$, $y_{\text{pred}}=\text{sigmoid}(3)$.

KEY CONCEPT

Answer 1:

$$\text{sigmoid}(3) = 1/(1+e^{-3}) = 1/(1+0.0498) = 0.9526$$

$$\text{gradient} = 0.9526*(1-0.9526) = 0.9526*0.0474 = 0.04516$$

$$\text{BCE} = -[1*\log(0.9526)+0*\log(0.0474)] = -\log(0.9526) = 0.0486$$

PRACTICE PROBLEM

Problem 2 (Ch 3): Weights $w=[3,-1]$. Loss $L=w_1^2+w_2^2$. Learning rate=0.2. Run 2 steps of gradient descent. What is L after 2 steps?

KEY CONCEPT

Answer 2:

$$dL/dw = [2w_1, 2w_2]$$

$$\text{Step 1: } g=[6, -2], w_{\text{new}}=[3-0.2*6, -1-0.2*(-2)]=[1.8, -0.6]$$

$$\text{Step 2: } g=[3.6, -1.2], w_{\text{new}}=[1.8-0.72, -0.6+0.24]=[1.08, -0.36]$$

$$L = 1.08^2 + 0.36^2 = 1.1664 + 0.1296 = 1.296$$

(Started at $L=10$, now 1.296 – significant reduction in 2 steps!)

PRACTICE PROBLEM

Problem 3 (Ch 4): Conv input 6x6, kernel 3x3, padding=1, stride=2. Find output size. If 32 kernels, how many params? (including bias)

KEY CONCEPT

Answer 3:

$$H_{\text{out}} = (6-3+2*1)/2 + 1 = 5/2 + 1 = 2.5+1 = 3.5 \rightarrow \text{floor} = 3$$

$$\text{Output: } 3 \times 3 \times 32$$

$$\text{Params} = 3*3*1*32 + 32 = 288 + 32 = 320$$

(Assuming single-channel input; multiply C_{in} if multi-channel)

PRACTICE PROBLEM

Problem 4 (Ch 5): LSTM: all $W=0.3$, $b=0$, $h_0=0$, $C_0=0$, $x=[1,2]$. Compute h_1 and h_2 (scalar LSTM). Use concatenated input $W*[h,x]$.

KEY CONCEPT

Answer 4:

$$t=1, x_1=1: \text{input_val} = 0.3*0+0.3*1=0.3$$

$$f_1=\sigma(0.3)=0.5744, i_1=\sigma(0.3)=0.5744, C_1=\tanh(0.3)=0.2913, o_1=\sigma(0.3)=0.5744$$

$$C_1 = 0.5744*0+0.5744*0.2913 = 0.1673$$

$$h_1 = 0.5744*\tanh(0.1673) = 0.5744*0.1662 = 0.0954$$

$$t=2, x_2=2: \text{input_val} = 0.3*0.0954+0.3*2=0.0286+0.6=0.6286$$

$$f_2=\sigma(0.6286)=0.6521, i_2=0.6521, C_2=\tanh(0.6286)=0.5577, o_2=0.6521$$

$$C_2 = 0.6521*0.1673+0.6521*0.5577 = 0.1091+0.3637 = 0.4728$$

$$h_2 = 0.6521*\tanh(0.4728) = 0.6521*0.4407 = 0.2873$$

PRACTICE PROBLEM

Problem 5 (Ch 6): Attention with $Q=[[1,1]]$, $K=[[1,0],[0,1],[1,1]]$, $V=[[5],[10],[15]]$, $d_k=2$. Compute attention output.

KEY CONCEPT

Answer 5:

$$\text{Scores} = Q*K^T/\text{sqrt}(2) = [1*1+1*0, 1*0+1*1, 1*1+1*1]/1.414$$

$$= [1, 1, 2]/1.414 = [0.707, 0.707, 1.414]$$

$$\text{exp: } [e^{0.707}, e^{0.707}, e^{1.414}] = [2.028, 2.028, 4.113], \text{ sum}=8.169$$

$$\text{attn} = [0.248, 0.248, 0.504]$$

$$\text{Output} = 0.248*5 + 0.248*10 + 0.504*15 = 1.24+2.48+7.56 = 11.28$$

PRACTICE PROBLEM

Problem 6 (Ch 7 — VAE): $\mu=[0,0]$, $\log_var=[0,0]$, $\epsilon=[1,-1]$. Compute z . What is KL loss?

KEY CONCEPT

Answer 6:

$$\text{sigma} = \exp(\log_var/2) = \exp(0) = [1,1]$$

$$z = \mu + \sigma \epsilon = [0, 0] + [1, 1] * [1, -1] = [1, -1]$$

$$KL = 0.5 * \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

$$= 0.5 * [(1+0-1-0) + (1+0-1-0)]$$

$$= 0.5 * [0+0] = 0$$

When $\mu=0$ and $\log_var=0$ ($\sigma=1$): $KL=0$. Latent distribution already matches $N(0, I)$!

COMPLETE FORMULA REFERENCE SHEET

Topic	Formula	Notes
L2 Norm	$\ x\ _2 = \sqrt{\sum x_i^2}$	Euclidean distance
Inner Product	$x^T y = \sum x_i y_i$	$= \ x\ \ y\ \cos(\theta)$
Hyperplane Dist	$d = (w^T x - w_0) / \ w\ $	Signed distance from hyperplane
Bayes	$P(X Y) = P(Y X)P(X)/P(Y)$	Prior $\times$ Likelihood / Evidence
KL Divergence	$KL(P Q) = \sum P \log(P/Q)$	Always ≥ 0
Cross-Entropy	$H(P;Q) = -\sum P \log(Q)$	$= KL+H(P)$; minimized in training
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$d\sigma/dx = \sigma(x)(1-\sigma(x))$
Tanh	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	$d/dx = 1 - \tanh^2(x)$
ReLU	$f(x) = \max(0, x)$	$df/dx = 1(x > 0)$
Softmax	$p_k = e^{z_k} / \sum e^{z_j}$	Multiclass probabilities
MSE Loss	$L = (1/N) \sum (y - \hat{y})^2$	Regression
BCE Loss	$L = -[y \log p + (1-y) \log(1-p)]$	Binary classification
CE Loss	$L = -\sum y_k \log p_k$	Multi-class
Normal Eq	$w^* = (X^T X)^{-1} X^T y$	Closed-form linear regression
GD Update	$w = w - \eta \text{grad}_L$	Gradient descent
Adam m	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$	1st moment ($\beta_1=0.9$)
Adam v	$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$	2nd moment ($\beta_2=0.999$)
Adam step	$w = w - \eta \cdot m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon)$	$\epsilon=1e-8$
L2 Reg	$L_{\text{total}} = L + \lambda \ w\ ^2/2$	Penalizes large weights
Batch Norm	$\hat{x} = (x - \mu) / (\sigma + \epsilon)$, $y = \gamma \hat{x} + \beta$	μ, σ over batch
Layer Norm	Same but μ, σ over features	Per-sample normalization
Conv Out Size	$H_{\text{out}} = (H - F + 2P) / S + 1$	F =filter, P =padding, S =stride
Conv Params	$F * F * C_{\text{in}} * C_{\text{out}} + C_{\text{out}}$	Per convolutional layer
RNN hidden	$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$	Shared weights across time
LSTM forget	$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$	Erases cell state
LSTM cell	$C_t = f_t * C_{t-1} + i_t * C_{\sim t}$	Long-term memory
LSTM hidden	$h_t = o_t * \tanh(C_t)$	Short-term output
GRU update	$h_t = u_t * h_{t-1} + (1-u_t) * \tilde{h}_t$	Interpolates old & new
Attention	$\text{softmax}(QK^T / \sqrt{d_k}) * V$	Scaled dot-product
PE sin	$PE(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$	Even dimensions
PE cos	$PE(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d})$	Odd dimensions
AE Loss	$L = \ x - \text{decoder}(\text{encoder}(x))\ ^2$	Reconstruction loss
VAE KL	$KL = 0.5 \sum (\sigma^2 + \mu^2 - 1 - \log \sigma^2)$	Force $z \sim N(0, I)$
VAE reparam	$z = \mu + \sigma * \epsilon$, $\epsilon \sim N(0, I)$	Makes sampling differentiable

Diffusion fwd	$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$	Closed form at step t
GCN	$H^k = \sigma(D^{-\frac{1}{2}} A D^{-\frac{1}{2}} H^{k-1} W^k)$	Graph convolution layer
Xavier init	$\text{std} = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$	For sigmoid/tanh
He init	$\text{std} = \sqrt{2/n_{\text{in}}}$	For ReLU

Every formula above has been derived and calculated in this document. If you understand the worked examples, you are exam-ready.

- Review backpropagation numerically at least once end-to-end.
- Know LSTM equations and be able to compute one time step.
- Practice conv output size and parameter count calculations.
- Understand KL divergence in VAE and why reparameterization is needed.